TURING CHALLENGE · PRUEBA TÉCNICA DE ARQUITECTURA

TechSolutions: API B2B de visión artificial en Azure

Atilio Camargo Moreira · 23 de octubre de 2026 · Código, Bicep y pipeline en el repositorio adjunto

La propuesta en cinco líneas

- **API asíncrona.** `POST` responde `202` con `jobId` en menos de 500 ms. Los 15 s de trabajo real van por colas, fuera de la conexión HTTP.
- **Entrada gobernada.** Front Door Premium con WAF delante de API Management Standard v2: un producto por tier, una suscripción por cliente, rate limit y cuota por contrato.
- **Cómputo por etapa.** Pre y postprocesado en Container Apps con KEDA. La inferencia, en un AKS pequeño con nodos T4 on-demand + Spot, escalado por profundidad de cola.
- **Datos.** PostgreSQL Flexible + PostGIS para los 150 M de parcelas, el estado de los jobs, el outbox y el ledger de uso. Resultados en Blob privado con lifecycle.
- **Coste base estimado:** ~3,4 k€/mes en valle y ~5,2 k€/mes en pico; unos **46 k€/año de Azure** (0,023 € por petición). Google Maps va aparte y pesa casi lo mismo.

1. Primero los números

Antes de elegir servicios hice las cuentas. Son las que deciden la arquitectura.

Dato	Cálculo e implicación
Tiempo de servicio nominal	2 + 10 + 3 = **15 s** por petición, sin cola ni reintentos.
Concurrencia a 50 RPS (Ley de Little)	50 × 15 = **750 peticiones en vuelo**. Ninguna API síncrona debería sostener eso con conexiones abiertas de 15 s.
Inferencias simultáneas a 50 RPS	50 × 10 = **500**. Si cada inferencia ocupa una T4 entera, son 500 GPUs: ~300 €/hora.
Volumen anual vs. picos	2 M/año = 0,063 RPS de media. A 50 RPS sostenidos, 2 M se agotan en **11,1 horas**.
Valles de 1–2 RPS continuos	1 RPS todo el año = 31,5 M peticiones. **No cuadra** con los 2 M anuales.
Almacenamiento de resultados	2 M × (5–30 KB + 0,4–1 MB) ≈ **0,8–2,1 TB/año**.
Catastro (150 M polígonos)	Con ~1–2 KB por geometría más índices GiST/B-tree estimo **200–350 GB**. De ahí 512 GB con autogrow.

> **Mi lectura:** los 50 RPS son ráfagas cortas en marzo–mayo y los 1–2 RPS son picos diarios, no carga continua. Dimensiono para absorber la ráfaga con colas y drenarla con capacidad elástica, no para procesar 50 RPS en tiempo real. Lo confirmaría con el cliente antes de comprar capacidad (sección 2).

Ejemplo de backlog que presentaría al cliente: si entran 50 RPS durante 10 minutos y la GPU procesa 10 RPS, se acumulan (50 − 10) × 600 = 24.000 jobs. Con 1 RPS de entrada posterior, se drenan a 9 RPS en unos 44 minutos. Ese es el trade-off real entre coste y latencia en pico, y es una decisión de negocio, no técnica.

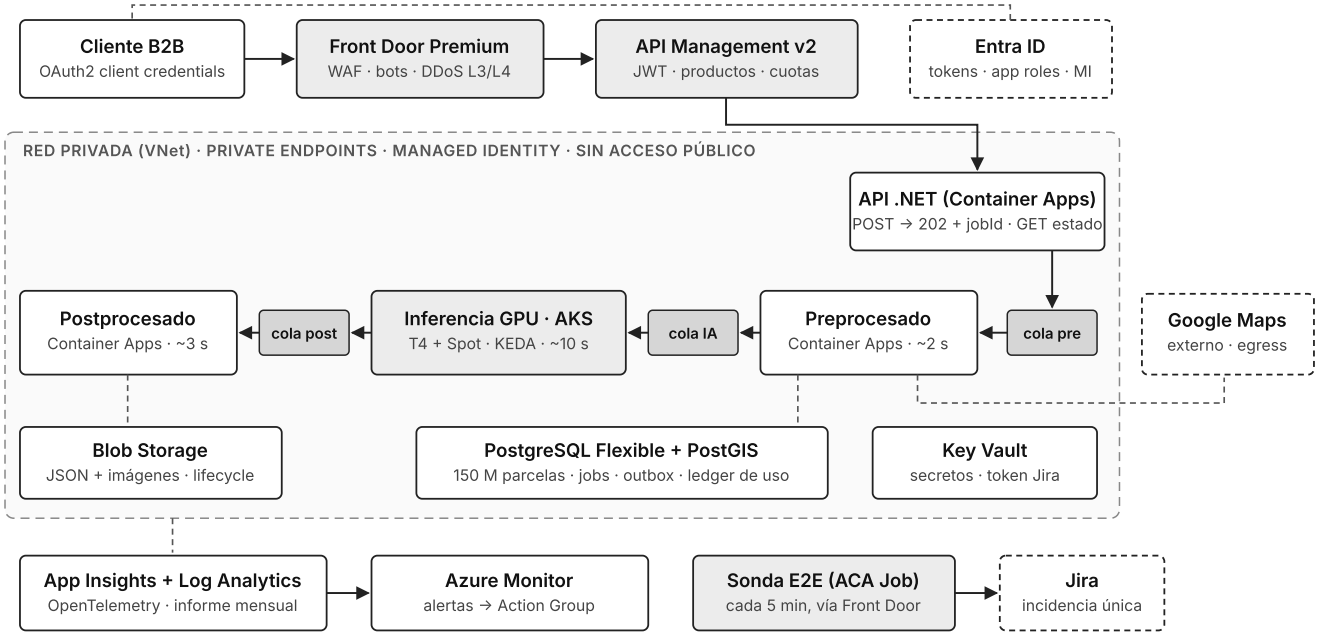
En un proyecto anterior bajamos una operación de ~60 s a ~5 s de respuesta percibida sacando el trabajo pesado del camino síncrono. Aquí el razonamiento es el mismo: el cliente necesita una confirmación rápida y un resultado fiable, no una conexión abierta 15 segundos.

2. Premisas y preguntas al cliente

Esta es la parte que haría en preventa. Cada respuesta cambia coste o SLA.

Pregunta	Mi premisa si no hay respuesta
¿Aceptáis integración asíncrona (polling o webhook)?	Sí. Ofrezco webhook firmado (HMAC) y polling.
¿Cuánto dura una ráfaga de 50 RPS y qué plazo máximo de resultado se contrata en pico?	Ráfagas ≤ 15 min; resultado ≤ 60 min en pico, p95 ≤ 60 s fuera de pico.
¿Dónde están los datos y los clientes? (el diagrama usa *sqft*; el catastro parece de EE. UU.)	West Europe. Si datos y clientes son de EE. UU., East US 2: menos latencia y GPU más barata.
¿Qué formato tiene el modelo y cuál es su throughput real por GPU? ¿Admite batch?	Contenedor PyTorch/ONNX; ~0,5 inferencias/s por T4 con batch (a validar en semana 1).
¿Qué permite la licencia de Google Maps sobre caché y almacenamiento de imágenes?	Sin caché permanente. Guardo el resultado del modelo, no la imagen original de Google, hasta revisarlo con legal.
¿Qué es un evento facturable: petición aceptada, inferencia o resultado entregado?	Resultado entregado (`completed`). Los fallos nuestros no se cobran.
¿Retención de resultados y obligación de borrado por cliente?	365 días, borrado por cliente bajo petición.
¿Disponibilidad, RPO/RTO y horario de soporte?	99,9 % de admisión; RPO 15 min, RTO 4 h; soporte 8×5 con guardia en temporada alta.

3. Arquitectura



Flujo principal. Cada etapa se comunica por cola (Service Bus) y persiste su resultado, de modo que un fallo no obliga a repetir llamadas a Google ni a la GPU.

Flujo de una petición

1. El cliente obtiene un token de Entra ID (client credentials) y llama a Front Door. El WAF filtra y Front Door reenvía solo a APIM.
2. APIM valida el JWT y el rol, aplica rate limit y cuota del producto, y toma el `tenantId` del token (nunca de una cabecera del cliente).
3. La API valida la petición, calcula un `jobId` determinista a partir de tenant + `Idempotency-Key`, guarda job y evento de outbox en la misma transacción y responde `202`.
4. El preprocesado consulta PostGIS (parcela por punto), descarga las imágenes de Google con timeout y circuit breaker, y guarda los artefactos en Blob.
5. La inferencia lee el artefacto, ejecuta el modelo versionado y deja la máscara en Blob.
6. El postprocesado convierte máscaras en polígonos, reduce vértices, calcula áreas y flags, y escribe el JSON final y la imagen resultante.
7. El job pasa a `completed`, se registra el evento facturable en el ledger y, si hay webhook, se notifica al cliente.

Estados: `accepted → preprocessing → inferencing → postprocessing → completed`, más `failed` (con código de error tipificado) y `cancelled`. Cada transición guarda versión de código y de modelo, intento, duración y coste estimado.

Contrato de la API

Operación	Respuesta
`POST /v1/evaluations`	`202 {jobId, status, statusUrl, submittedAt}`. Cabecera `Idempotency-Key`: el mismo valor devuelve el mismo job. Errores 400/401/403/413/429.
`GET /v1/evaluations/{jobId}`	`200 {status, progress, error?}`. Siempre filtrado por tenant: un job ajeno responde 404.

`GET /v1/evaluations/{jobId}/result`	`200` con polígonos, sqft por polígono, lat/long, flags y código de error; `409` si aún no está listo.
`GET /v1/usage`	Volumen aceptado, completado, fallido y facturable por periodo, desde el ledger.

4. Requisitos no funcionales y cómo los cubro

Requisito	Objetivo propuesto	Decisión que lo cumple
Latencia de admisión	p95 < 500 ms	La API solo valida y encola. Escala por peticiones concurrentes (mín. 2 réplicas).
Tiempo hasta resultado	p95 ≤ 60 s fuera de pico	Colas por etapa con escalado KEDA; en pico, plazo pactado según duración de ráfaga.
Picos de 50 RPS	0 peticiones perdidas	Service Bus absorbe la ráfaga; pool GPU Spot hasta 16 nodos; precalentado antes de marzo.
Disponibilidad	99,9 % admisión	Zonas de disponibilidad en ACA, Service Bus, PostgreSQL HA y ACR; Front Door global.
Seguridad	Sin secretos ni endpoints públicos de datos	Managed Identity en todo; Private Endpoints; Key Vault; Entra ID sin contraseñas en PostgreSQL.
Multi-tenant y cuota	Aislamiento y uso medible por cliente	Suscripción APIM por cliente, tenant desde el token, filtros por tenant, ledger de uso idempotente.
Trazabilidad	Estadística mensual de tiempos, volumen y errores	OpenTelemetry con jobId/tenant; métricas por etapa; informe mensual (KQL en el repo); 90 días + export mensual.
Operación	Detección antes que el cliente	Sonda E2E cada 5 min con incidencia en Jira; alertas por etapa; DLQ con reconciliación.
Coste	Mínimo fuera de temporada	Escalado por cola; 1 GPU caliente en valle; Spot en pico; presupuesto con alertas al 70/90/100 %.
Recuperación	RPO 15 min · RTO 4 h	Backups PITR de PostgreSQL; ZRS en Blob; infraestructura reproducible con Bicep.

5. Datos: catastro, estado y resultados

PostgreSQL Flexible Server + PostGIS (General Purpose D4ds_v5, HA entre zonas, 512 GB). La consulta crítica es espacial: dada una coordenada, encontrar la parcela (`ST_Intersects` con índice GiST). Es exactamente lo que PostGIS resuelve bien y lo que Cosmos DB no resuelve sin trabajo extra. La carga inicial va por lotes con `COPY` sobre una tabla de staging, se crean índices al final y se valida el plan con `EXPLAIN ANALYZE`. Particionaría por región solo si el benchmark lo justifica.

En la misma base guardo el estado de los jobs, el outbox y el ledger de uso, en esquemas separados. Así, aceptar un job y registrar su evento es una sola transacción, sin coordinación distribuida. Si el volumen crece mucho, el ledger es lo primero que separaría.

Blob Storage (ZRS, sin acceso público, sin claves compartidas, solo Entra ID). Contenedores `results` e `images` con ruta `{tenant}/{yyyy}/{MM}/{jobId}`. Lifecycle: Cool a los 30 días, Cold a los 90 y borrado a los 365 (configurable por contrato). Los artefactos intermedios se borran a los 7 días. Para descargas, SAS de delegación de usuario con 15 minutos de validez.

6. Escalabilidad e inferencia

Separo el escalado de CPU y de GPU porque tienen costes y tiempos de arranque muy distintos. La métrica que manda en todas las etapas es la profundidad de cola, no la CPU: la CPU no dice si vamos a cumplir el plazo.

Componente	Dónde	Regla de escalado	Límites y motivo
API	Container Apps	50 peticiones concurrentes por réplica	2–10 réplicas. Trabajo ligero; 2 mínimas por disponibilidad.
Preprocesado	Container Apps (KEDA)	10 mensajes por réplica	1–30. El máximo protege PostGIS y la cuota de Google.
Inferencia	AKS, nodos T4 (KEDA + cluster autoscaler)	Pods por mensajes pendientes; 1 pod por GPU	Valle: 1 nodo caliente. Pico: 2–4 on-demand + hasta 16 Spot.
Postprocesado	Container Apps (KEDA)	10 mensajes por réplica	1–20. CPU intensivo (máscaras a polígonos).

Por qué AKS para la GPU

Para la inferencia elegí un AKS pequeño, solo con este cometido. Me da tres cosas que necesito en pico: control del batching (Triton u ONNX Runtime), un pool Spot para la ráfaga (un desalojo solo retrasa el job, porque el mensaje vuelve a la cola) y cuota de GPU predecible por familia de VM. Descarté Container Apps serverless GPU porque su cuota por defecto es baja para cientos de inferencias en paralelo, y Azure ML Online Endpoints porque añade una plataforma más sin resolver el problema de capacidad. Lo reconsideraría si el equipo ya opera Azure ML.

En BASF, con Zühlke, escalamos workers en AKS con KEDA según la profundidad de la cola. Es el mismo patrón y conozco bien sus puntos débiles: tiempo de arranque de nodos GPU (3–5 min), descarga de imágenes grandes y presupuestos de disrupción.

Preparación de la temporada alta

- **Enero:** solicitud de cuota GPU en la región y prueba de carga con ráfagas de 5, 10 y 30 minutos.
- **Febrero:** subo `gpuMinNodes` a 2 en el fichero de parámetros de producción y escalo PostgreSQL a D8ds_v5.
- **Junio:** vuelta al perfil de valle. Son cambios de parámetro en Bicep revisados por pull request, no tareas manuales.

7. Seguridad, multi-tenant y cuotas

Identidad. Cada cliente tiene su app en Entra ID con el rol `TechSolutions.Submit` y obtiene tokens por client credentials (certificado recomendado). APIM valida el token y la API lo vuelve a validar (defensa en profundidad). El tenant sale del claim `azp` del token, así que un cliente no puede hacerse pasar por otro cambiando una cabecera.

Cuotas y facturación. Un producto APIM por tier y una suscripción por contrato:

Producto	Rate limit	Cuota (30 días)	Uso típico
Bronze	2 envíos/s	20.000 jobs	Cliente pequeño o piloto
Silver	10 envíos/s	200.000 jobs	Cliente medio
Gold	50 envíos/s	1.000.000 jobs	Gran cuenta con temporada alta

La cuota de APIM protege la plataforma, pero no es una factura: se reinicia cada 30 días desde la primera llamada y no conoce el resultado del job. La factura sale del **ledger** en PostgreSQL: un evento por job y tenant,

idempotente, con el estado final. APIM emite además una métrica por job aceptado (tenant y producto como dimensiones) para el panel de uso y para conciliar.

Red y perímetro. Front Door Premium incluye protección DDoS de capa 3/4. El WAF añade reglas OWASP gestionadas, protección contra bots y un rate limit grueso por IP. APIM solo acepta tráfico de nuestro Front Door (cabecera `X-Azure-FDID`), y el entorno de Container Apps es interno. PostgreSQL, Storage, Service Bus, Key Vault y ACR no tienen endpoint público. Como no hay IPs públicas propias detrás de Front Door, no necesito DDoS Network Protection, que costaría ~2,7 k€/mes.

Gobierno. Managed Identity en todos los servicios, roles RBAC mínimos, PIM para accesos humanos, Azure Policy (sin acceso público, etiquetas obligatorias), Defender for Cloud y logs de auditoría centralizados.

---

8. Observabilidad, sonda E2E y alertas

Instrumento las piezas .NET con OpenTelemetry hacia Application Insights. El `traceId` viaja en los mensajes de Service Bus, así que una petición se sigue de punta a punta. Cada etapa emite `stage.duration` y el cierre del job emite `job.e2e.duration`, con tenant y resultado. El **informe mensual** (peticiones, completadas, errores, p95 de admisión, tiempos medios por etapa y por cliente) es una consulta KQL incluida en el repositorio, publicada como Workbook y exportada a Blob cada mes. La retención es de 90 días; el histórico anual sale de esos exports y del ledger.

Sonda sintética. Un Container Apps Job se ejecuta cada 5 minutos con una identidad gestionada que tiene el rol de la API, así que no necesita secreto de cliente. Entra por Front Door como un cliente más y envía una parcela con resultado conocido. Hace polling hasta el estado final y compara con tolerancia: área ±2 %, mismo número de polígonos, mismos flags y versión de modelo informada. Si el resultado es incorrecto, no hay respuesta o se supera el timeout en dos intentos seguidos, abre una incidencia en Jira. Si ya hay una abierta con la etiqueta `synthetic-e2e`, añade un comentario en vez de duplicarla. Si Jira no responde, la ejecución fallida dispara igualmente la alerta de Azure Monitor.

Alerta	Condición inicial	Por qué
Sonda E2E fallida	Cualquier fallo en 15 min (sev. 1)	El cliente está viendo un error ahora.
Sonda E2E sin datos	Sin ejecuciones en 20 min	El monitor también puede caerse.
Dead-letter	> 0 mensajes en cualquier cola	Un job agotó reintentos; hay que reconciliar.
Backlog de inferencia	> 5.000 mensajes durante 15 min	La GPU no alcanza el plazo pactado.
Errores 5xx	> 2 % con ≥ 50 peticiones en 5 min	Degradación de la API.
Latencia de admisión	p95 > 500 ms durante 10 min	La admisión debe ser siempre rápida.
PostGIS CPU	> 80 % durante 15 min	Si el catastro va lento, todo va lento.
Presupuesto	70 % y 90 % real, 100 % previsto	Una GPU mal escalada se nota primero en la factura.

---

9. CI/CD con Azure DevOps

Un pipeline YAML multi-stage (en el repositorio) con el principio **build once, deploy many**: la imagen etiquetada con el commit que pasa en dev es exactamente la que llega a producción.

- **CI (cada PR):** build, tests con cobertura, bloqueo por paquetes NuGet vulnerables (High/Critical), escaneo de secretos (gitleaks), SBOM CycloneDX y lint de Bicep. En `main`, imágenes construidas en ACR con `az acr build`.
- **Dev:** `what-if` de Bicep, despliegue idempotente y ejecución de la sonda E2E como smoke test.
- **Producción:** aprobación manual en el environment de ADO (con el `what-if` en el log para el aprobador), importación de las imágenes desde el ACR de dev, canary del 10 % en una nueva revisión de Container

Apps, gate automático por tasa de 5xx durante 15 minutos, promoción al 100 % y sonda E2E.

- **Seguridad del pipeline:** service connections con Workload Identity Federation (OIDC), sin secretos guardados; ramas protegidas y revisión obligatoria.
- **Rollback:** devolver el tráfico a la revisión anterior tarda segundos. Los modelos se promocionan igual que el código, con un gate de calidad sobre un dataset de referencia etiquetado.

10. Decisiones de arquitectura

Decisión	Elegido	Descartado y por qué
Interacción	Asíncrona: 202 + polling o webhook	Síncrona: 750 conexiones de 15 s en pico; timeouts y reintentos en cascada.
Catastro	PostgreSQL + PostGIS	Cosmos DB: las consultas espaciales sobre polígonos son limitadas y caras en RU. Azure SQL tiene tipos espaciales, pero PostGIS es más completo y más barato.
Inferencia	AKS con pool T4 + Spot	ACA serverless GPU: cuota baja para el pico. Azure ML endpoint: otra plataforma sin ganancia en capacidad.
Pre y postproceso	Container Apps + KEDA	Ejecutarlos en AKS: más operación sin beneficio; aquí no hace falta GPU.
Mensajería	Service Bus Premium	Storage Queues: sin sesiones, sin deduplicación ni DLQ nativa. Event Grid: pensado para eventos, no para colas de trabajo.
Gateway	APIM Standard v2	Premium (clásico o v2): ~4× el coste. Solo lo necesitaría con multi-región activa o inyección completa en VNet.
Sonda E2E	Container Apps Job (cron)	Azure Function: igual de válida, pero añade otro runtime y otro pipeline. El Job reutiliza imagen, identidad y despliegue.
Consistencia	Outbox transaccional + IDs deterministas	Publicar en la cola dentro de la petición sin outbox: jobs huérfanos o duplicados.

11. Coste mensual estimado

Precios de lista pay-as-you-go en West Europe, redondeados. Es un orden de magnitud para decidir; lo cerraría con la Pricing Calculator y la suscripción real.

Componente	Valle (jun–feb)	Pico (mar–may)
Front Door Premium + WAF	320 €	340 €
API Management Standard v2 (1 unidad)	650 €	650 €
Container Apps (API, workers, sonda)	150 €	400 €
Service Bus Premium (1 MU)	625 €	625 €
PostgreSQL Flexible D4ds_v5 HA, 512 GB (D8 en pico)	700 €	1.200 €
AKS: plano de control Standard + 2 nodos de sistema	230 €	230 €
GPU T4 (valle: 1 nodo; pico: 2 on-demand + Spot)	440 €	1.300 €
Storage, ACR Premium, Key Vault, Private Endpoints y DNS	150 €	175 €
Log Analytics, Application Insights y alertas	120 €	250 €
Total Azure	**≈ 3.390 €**	**≈ 5.170 €**

Anual: 9 × 3.390 + 3 × 5.170 ≈ **46 k€**, unos 0,023 € por petición.

Google Maps (aparte): entre 9 y 14 USD por 1.000 jobs (Static Maps satélite + Street View Static, más geocodificación si el catastro no da coordenadas). Con 2 M de jobs son **18–28 k USD/año**, del mismo orden que toda la infraestructura Azure. Por eso persisto los artefactos del preprocesado: un reintento nunca vuelve a pagar a Google.

Optimización

- Reservas a 1 año para PostgreSQL, nodos de sistema y la T4 base: ~−450 €/mes.
- GPU a 0 por la noche en valle, si el SLA acepta 3–5 min de arranque: ~−175 €/mes.
- PostgreSQL a D2ds_v5 en valle si el benchmark lo permite: ~−250 €/mes.
- Logs: tablas Basic para logs verbosos y muestreo de trazas. Las métricas de negocio no se muestrean.
- Service Bus Standard ahorraría ~600 €/mes, pero pierde los private endpoints. No lo recomiendo con clientes enterprise.

Con las tres primeras medidas el año baja a unos **36–38 k€**. La sensibilidad que pondría delante del cliente es otra: procesar 50 RPS en tiempo real, sin cola, con una T4 por inferencia, cuesta ~300 € por hora de ráfaga. Antes de prometer un p95 en pico, hay que medir el modelo.

---

12. Plan de entrega y equipo

Fase	Semanas	Entregable y criterio de salida
Discovery + benchmark	1–2	Respuestas del cliente, throughput real del modelo por GPU, carga de una muestra del catastro con p95 de consulta, cuota GPU solicitada.
MVP extremo a extremo	3–5	API, colas, tres etapas, PostGIS, Blob, auth y APIM con un cliente piloto. Observabilidad básica y CI/CD a dev.
Hardening	6–8	Red privada completa, sonda + Jira, alertas, ledger y facturación, pruebas de carga, caos y restauración, evaluación del modelo.
Producción controlada	9–10	Canary con dos clientes, runbooks, revisión de seguridad, coste real frente a estimado.

Equipo: 1 arquitecto / technical PM (yo), 2 backend .NET, 1 platform / DevOps, 1 ML engineer, QA compartido y apoyo puntual de GIS y seguridad. El camino crítico no es el código: es la cuota de GPU, la carga de los 150 M de parcelas, la licencia de Google y la calidad del modelo. Por eso van en la semana 1.

---

13. Riesgos y mitigación

Riesgo	Impacto	Mitigación
Sin capacidad GPU en la región durante el pico	Backlog y plazos incumplidos	Cuota pedida en enero, familia alternativa (A10/A100), pool Spot y región secundaria preparada en Bicep.
El modelo rinde menos de lo previsto por GPU	Coste de GPU ×2–×5	Benchmark en semana 1; batching; cuantización o ONNX; renegociar plazo en pico.
Google Maps cae, limita o sube precios	Errores o margen negativo	Circuit breaker, reintentos acotados, cuota por cliente, coste por job vigilado, proveedor alternativo evaluado.
Licencia de Google impide guardar imágenes	Riesgo contractual	Revisión legal antes de producción; guardar solo resultados derivados.
Jobs duplicados o perdidos	Doble cobro o resultados que faltan	IDs deterministas, outbox, deduplicación de Service Bus, DLQ con reconciliación diaria.

Fuga de datos entre clientes	Incidente contractual grave	Tenant desde el token, filtros en cada consulta y ruta, tests de acceso cruzado en CI.
Regresión del modelo	Resultados incorrectos sin errores visibles	Dataset de referencia, gate de calidad, sonda con tolerancias, rollback de versión.

14. Lo que haría distinto con más tiempo

- Benchmark real del modelo en T4 y A10 antes de fijar SKU; hoy el throughput por GPU es la premisa más débil del documento.
- Prueba de carga del catastro con datos reales: tamaño, índices y p95 de la consulta punto-en-polígono.
- Prototipo del webhook firmado y de la reconciliación de DLQ, que en el repositorio solo están descritos.
- Un Workbook completo como código y un runbook por alerta.
- Análisis de región con el cliente: si los datos son de EE. UU., rehacer el coste en East US 2.

Repositorio

API .NET 10 (contrato, idempotencia, aislamiento por tenant), worker de referencia, sonda E2E con integración Jira y 19 tests automatizados; Bicep modular (red, datos, PostgreSQL, Container Apps, AKS GPU, APIM con productos, Front Door + WAF, alertas y presupuesto) con parámetros de dev y prod; políticas APIM; pipeline Azure DevOps multi-stage; consulta KQL del informe mensual.

Referencias

Microsoft Learn: API Management *quota-by-key* y *rate-limit-by-key*; arquitectura multi-tenant con API Management; Container Apps (escalado KEDA, Jobs, entornos internos); AKS con nodos GPU y complemento KEDA; PostgreSQL Flexible Server (extensiones, PostGIS); Front Door + WAF; Application Insights. Google Maps Platform: precios por SKU (marzo 2025) y términos de servicio sobre caché.